
Algorithm 1: Convolutional Autoencoder with Constant Padding

```
Data: Log-Mel spectrogram  $\mathbf{X} \in \mathbb{R}^{1 \times 64 \times 313}$ 
Result: Reconstructed spectrogram  $\hat{\mathbf{X}}$ 
//Hyper-parameter
latent_dim  $\leftarrow$  256

//Encoder
 $\mathbf{H} \leftarrow \text{Conv2D}(1, 16, 3, 1) \circ \text{ReLU} \circ \text{MaxPool2D}(2)(\mathbf{X})$ 
 $\mathbf{H} \leftarrow \text{Conv2D}(16, 32, 3, 1) \circ \text{ReLU} \circ \text{MaxPool2D}(2)(\mathbf{H})$ 
 $\mathbf{H} \leftarrow \text{Conv2D}(32, 64, 3, 1) \circ \text{ReLU} \circ \text{MaxPool2D}(2)(\mathbf{H})$  //output size  $64 \times 8 \times 39$ 

//Fully-connected bottleneck
 $\mathbf{z} \leftarrow \text{Linear}(64 \cdot 8 \cdot 39, \text{latent\_dim}) \circ \text{Flatten}(\mathbf{H})$ 

//Decoder (inverse FC)
 $\mathbf{H}' \leftarrow \text{Unflatten} \circ \text{Linear}(\text{latent\_dim}, 64 \cdot 8 \cdot 39)(\mathbf{z})$ 

//Decoder (deconvolutions)
 $\mathbf{H}' \leftarrow \text{ConvTr2D}(64, 32, 4, 2, 1) \circ \text{ReLU}(\mathbf{H}')$ 
 $\mathbf{H}' \leftarrow \text{ConvTr2D}(32, 16, 4, 2, 1) \circ \text{ReLU}(\mathbf{H}')$ 
 $\hat{\mathbf{X}} \leftarrow \text{ConvTr2D}(16, 1, 4, 2, 1)(\mathbf{H}')$ 

//Final padding to width 313
 $\hat{\mathbf{X}} \leftarrow \text{ConstantPad2D}((0, 1, 0, 0), 0)(\hat{\mathbf{X}})$ 
```

Algorithm 2: SVDDNet — Convolutional Encoder for Deep-SVDD

```
Data: Mini-batch of log-Mel spectrograms  $\mathbf{X} \in \mathbb{R}^{B \times 1 \times H \times W}$ 
Result: Embedding matrix  $\mathbf{Z} \in \mathbb{R}^{B \times \text{emb\_dim}}$ 
//Hyper-parameter
emb_dim  $\leftarrow$  128

//Constructor (_init_)
define conv as:
    Conv2D(1, 16, 3, pad = 1)  $\rightarrow$  ReLU  $\rightarrow$  MaxPool2D(2)
    Conv2D(16, 32, 3, pad = 1)  $\rightarrow$  ReLU  $\rightarrow$  MaxPool2D(2)
    Flatten

//Compute the flattened dimension dynamically
 $\mathbf{X}_{\text{dummy}} \leftarrow$  one batch from train_loader
 $\mathbf{h} \leftarrow \text{conv}(\mathbf{X}_{\text{dummy}})$ 
flat_dim  $\leftarrow \text{dim}(\mathbf{h}, 1)$ 
define fc  $\leftarrow \text{Linear}(\text{flat\_dim}, \text{emb\_dim})$ 

Function forward( $\mathbf{X}$ )
     $\mathbf{h} \leftarrow \text{conv}(\mathbf{X})$ 
     $\mathbf{Z} \leftarrow \text{fc}(\mathbf{h})$ 
    return  $\mathbf{Z}$ 
```
